Data Structures

HomeWork Set -5

1. Use a linked list for polynomial representation and perform addition and subtraction operations on the polynomials.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int coeff, exp;
   struct Node *next;
};

struct Node *createNode(int c, int e) {
   struct Node *n = malloc(sizeof(struct Node));
   n->coeff = c;
   n->exp = e;
   n->next = NULL;
   return n;
}

void insert(struct Node **head, int c, int e) {
   if (c == 0)
      return;

   struct Node *p = *head, *prev = NULL;

   while (p != NULL && p->exp > e) {
      prev = p;
      p = p->next;
   }

   if (p != NULL && p->exp == e) {
      p->coeff += c;
      if (p->coeff == 0) {
         if (prev != NULL)
            prev->next = p->next;
         else
            *head = p->next;
         free(p);
      }
      return;
   }
```

```
    struct Node *n = createNode(c, e);
    n->next = p;

    if (prev != NULL)
        prev->next = n;
    else
        *head = n;
}

struct Node *readPolynomial(void) {
    struct Node *head = NULL;
    int n, c, e;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter coefficient and exponent: ");
        scanf("%d%d", &c, &e);
        insert(&head, c, e);
    }

    return head;
}

struct Node *calculate(struct Node *a,
                       struct Node *b, int sign) {
    struct Node *result = NULL;

    while (a != NULL) {
        insert(&result, a->coeff, a->exp);
        a = a->next;
    }

    while (b != NULL) {
        insert(&result, sign * b->coeff, b->exp);
        b = b->next;
    }

    return result;
}

void display(struct Node *p) {
    if (p == NULL) {
        printf("0");
        return;
    }

    while (p != NULL) {
        printf("%dx^%d", p->coeff, p->exp);
```

```
        p = p->next;

        if (p != NULL)
            printf(" + ");
    }
}

int main(void) {
    struct Node *p1, *p2, *sum, *diff;

    printf("First polynomial:\n");
    p1 = readPolynomial();

    printf("Second polynomial:\n");
    p2 = readPolynomial();

    sum = calculate(p1, p2, 1);
    diff = calculate(p1, p2, -1);

    printf("\nAddition: ");
    display(sum);

    printf("\nSubtraction: ");
    display(diff);

    printf("\n");
    return 0;
}
```

Output :

```
First polynomial:
Enter number of terms: 3
Enter coefficient and exponent: 5 3
Enter coefficient and exponent: 4 2
Enter coefficient and exponent: 2 0
Second polynomial:
Enter number of terms: 3
Enter coefficient and exponent: 3 3
Enter coefficient and exponent: 2 1
Enter coefficient and exponent: -1 0
Addition: 8x^3 + 4x^2 + 2x^1 + 1x^0
Subtraction: 2x^3 + 4x^2 + -2x^1 + 3x^0
```

2. Write a C program to find the previous node of a given node in a circular linked list.

```
#include <stdio.h>
```

```c
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main(void) {
    struct Node *head = NULL, *tail = NULL, *newNode;
    struct Node *current, *previous;
    int n, value, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &value);

        newNode = malloc(sizeof(struct Node));
        newNode->data = value;

        if (head == NULL) {
            head = tail = newNode;
            newNode->next = head;
        } else {
            newNode->next = head;
            tail->next = newNode;
            tail = newNode;
        }
    }

    printf("Enter key: ");
    scanf("%d", &key);

    if (head == NULL) {
        printf("List is empty\n");
        return 0;
    }

    current = head;
    previous = tail;

    do {
        if (current->data == key) {
            printf("Previous node: %d\n", previous->data);
            return 0;
        }

        previous = current;
        current = current->next;
```

```
    } while (current != head);

    printf("Key not found\n");

    return 0;
}
```

Output :

```
Enter number of nodes: 5
10 20 30 40 50
Enter key: 10
Previous node: 50
```

3. Write a C program to insert a node at a given position in a circular linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main(void) {
    struct Node *head = NULL, *tail = NULL;
    struct Node *newNode, *current;
    int n, value, pos;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &value);

        newNode = malloc(sizeof(struct Node));
        newNode->data = value;

        if (head == NULL) {
            head = tail = newNode;
            newNode->next = head;
        } else {
            newNode->next = head;
            tail->next = newNode;
            tail = newNode;
        }
```

```
    }

    printf("Enter value and position: ");
    scanf("%d%d", &value, &pos);

    if (pos < 1 || pos > n + 1) {
        printf("Invalid position\n");
        return 0;
    }

    newNode = malloc(sizeof(struct Node));
    newNode->data = value;

    if (pos == 1) {
        if (head == NULL) {
            head = tail = newNode;
            newNode->next = head;
        } else {
            newNode->next = head;
            head = newNode;
            tail->next = head;
        }
    } else {
        current = head;

        for (int i = 1; i < pos - 1; i++)
            current = current->next;

        newNode->next = current->next;
        current->next = newNode;

        if (pos == n + 1)
            tail = newNode;
    }

    printf("Circular linked list: ");

    current = head;

    if (current != NULL) {
        do {
            printf("%d ", current->data);
            current = current->next;
        } while (current != head);
    }

    printf("\n");
    return 0;
}
```

Output :

```
Enter number of nodes: 4
10 20 30 40
Enter value and position: 25 3
Circular linked list: 10 20 25 30 40
```

4.Write a C program to find the second-largest element in a queue.

```
#include <stdio.h>
#include <limits.h>

#define MAX 100

int main(void) {
    int queue[MAX];
    int n, largest = INT_MIN;
    int second = INT_MIN;

    printf("Enter queue size: ");
    scanf("%d", &n);

    if (n < 2 || n > MAX) {
        printf("Invalid queue size\n");
        return 0;
    }

    printf("Enter queue elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &queue[i]);
    }

    for (int i = 0; i < n; i++) {
        if (queue[i] > largest) {
            second = largest;
            largest = queue[i];
        } else if (queue[i] < largest && queue[i] > second) {
            second = queue[i];
        }
    }

    if (second == INT_MIN)
        printf("Second-largest distinct element does not exist\n");
    else
        printf("Second-largest element: %d\n", second);

    return 0;
}
```

Output :

```
Enter queue size: 6
Enter queue elements: 10 40 20 50 30 50
Second-largest element: 40
```

5.Write a C program to insert a node at the third position in a linked list and check whether the first and third elements are the same.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

int main(void) {
    struct Node *head = NULL, *tail = NULL;
    struct Node *newNode, *current;
    int n, value, insertValue;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        scanf("%d", &value);

        newNode = malloc(sizeof(struct Node));
        newNode->data = value;
        newNode->next = NULL;

        if (head == NULL) {
            head = tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }
    }

    if (n < 2) {
        printf("At least two nodes are required\n");
        return 0;
    }

    printf("Enter value to insert at position 3: ");
    scanf("%d", &insertValue);

    newNode = malloc(sizeof(struct Node));
    newNode->data = insertValue;
```

```
    current = head;

    for (int i = 1; i < 2; i++) {
        current = current->next;
    }

    newNode->next = current->next;
    current->next = newNode;

    printf("Linked list: ");

    current = head;
    while (current != NULL) {
        printf("%d ", current->data);
        current = current->next;
    }

    if (head->data == head->next->next->data)
        printf("\nFirst and third elements are same\n");
    else
        printf("\nFirst and third elements are different\n");

    return 0;
}
```

Output :

```
Enter number of nodes: 4
10 20 10 40
Enter value to insert at position 3: 30
Linked list: 10 20 30 10 40
First and third elements are different
```

6. Implement two stacks in a single array. Write a routine to traverse the array and display its elements.

```
#include <stdio.h>
#define SIZE 10

int arr[SIZE];
int top1 = -1;
int top2 = SIZE;

void push1(int value) {
    if (top1 + 1 == top2) {
        printf("Stack Overflow\n");
    } else {
        arr[++top1] = value;
    }
```

```
}

void push2(int value) {
    if (top1 + 1 == top2) {
        printf("Stack Overflow\n");
    } else {
        arr[--top2] = value;
    }
}

void display(void) {
    printf("Array elements: ");

    for (int i = 0; i < SIZE; i++) {
        if (i <= top1 || i >= top2)
            printf("%d ", arr[i]);
        else
            printf("_ ");
    }

    printf("\n");
}

void displayStacks(void) {
    printf("Stack 1 (bottom to top): ");

    for (int i = 0; i <= top1; i++)
        printf("%d ", arr[i]);

    printf("\nStack 2 (bottom to top): ");

    for (int i = SIZE - 1; i >= top2; i--)
        printf("%d ", arr[i]);

    printf("\n");
}

int main(void) {
    push1(10);
    push1(20);
    push1(30);

    push2(90);
    push2(80);
    push2(70);

    displayStacks();
    display();

    return 0;
```

}

Output :

Stack 1 (bottom to top): 10 20 30
Stack 2 (bottom to top): 90 80 70
Array elements: 10 20 30 _ _ _ _ 70 80 90

7.Given a linked list where each node contains two pointers:

next — points to the next main-list node.

bottom — points to a sorted sub-linked list.

Flatten the linked list into a single sorted linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
    struct Node *bottom;
};

struct Node *createNode(int data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->next = NULL;
    n->bottom = NULL;
    return n;
}

struct Node *merge(struct Node *a, struct Node *b) {
    if (a == NULL)
        return b;

    if (b == NULL)
        return a;

    struct Node *result;

    if (a->data <= b->data) {
        result = a;
        result->bottom = merge(a->bottom, b);
    } else {
        result = b;
        result->bottom = merge(a, b->bottom);
```

```
    }

    result->next = NULL;
    return result;
}

struct Node *flatten(struct Node *head) {
    if (head == NULL || head->next == NULL)
        return head;

    head->next = flatten(head->next);

    head = merge(head, head->next);

    return head;
}

void display(struct Node *head) {
    while (head != NULL) {
        printf("%d", head->data);

        head = head->bottom;

        if (head != NULL)
            printf(" -> ");
    }

    printf(" -> NULL\n");
}

int main(void) {
    struct Node *head = createNode(5);

    head->next = createNode(10);
    head->next->next = createNode(19);
    head->next->next->next = createNode(28);

    head->bottom = createNode(7);
    head->bottom->bottom = createNode(8);

    head->next->bottom = createNode(20);

    head->next->next->bottom = createNode(22);

    head->next->next->next->bottom = createNode(40);
    head->next->next->next->bottom->bottom = createNode(45);

    head = flatten(head);

    printf("Flattened linked list:\n");
```

```
    display(head);

    return 0;
}
```

Output :

```
Flattened linked list:
5 -> 7 -> 8 -> 10 -> 19 -> 20 -> 22 -> 28 -> 40 -> 45 -> NULL
```

8.Given k sorted linked lists of different sizes, write a C program to merge them into a single linked list while maintaining sorted order.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *createNode(int data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->next = NULL;
    return n;
}

void insertEnd(struct Node **head, struct Node **tail, int data) {
    struct Node *n = createNode(data);

    if (*head == NULL) {
        *head = *tail = n;
    } else {
        (*tail)->next = n;
        *tail = n;
    }
}

int main(void) {
    int k, n, value;
    struct Node **head, **tail;
    struct Node *result = NULL, *last = NULL, *temp;

    printf("Enter number of sorted lists: ");
    scanf("%d", &k);

    if (k <= 0) {
        printf("Invalid number of lists\n");
        return 0;
```

```
}

head = calloc(k, sizeof(struct Node *));
tail = calloc(k, sizeof(struct Node *));

if (head == NULL || tail == NULL) {
   printf("Memory allocation failed\n");
   return 1;
}

for (int i = 0; i < k; i++) {
   printf("Enter size of list %d: ", i + 1);
   scanf("%d", &n);

   printf("Enter sorted elements: ");

   for (int j = 0; j < n; j++) {
      scanf("%d", &value);
      insertEnd(&head[i], &tail[i], value);
   }
}

while (1) {
   int minIndex = -1;

   for (int i = 0; i < k; i++) {
      if (head[i] != NULL &&
         (minIndex == -1 ||
          head[i]->data < head[minIndex]->data)) {
         minIndex = i;
      }
   }

   if (minIndex == -1)
      break;

   temp = head[minIndex];
   head[minIndex] = head[minIndex]->next;
   temp->next = NULL;

   if (result == NULL) {
      result = last = temp;
   } else {
      last->next = temp;
      last = temp;
   }
}

printf("Merged linked list: ");
```

```
    temp = result;
    while (temp != NULL) {
        printf("%d", temp->data);
        temp = temp->next;

        if (temp != NULL)
            printf(" -> ");
    }

    printf(" -> NULL\n");

    return 0;
}
```

Output :

```
Enter number of sorted lists: 3
Enter size of list 1: 3
Enter sorted elements: 1 4 7
Enter size of list 2: 4
Enter sorted elements: 2 5 8 10
Enter size of list 3: 2
Enter sorted elements: 3 6
Merged linked list: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 10 -> NULL
```

9. Write a C program to swap the \(k^{th}\) node from the beginning with the \(k^{th}\) node from the end in a linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *createNode(int data) {
    struct Node *n = malloc(sizeof(struct Node));
    n->data = data;
    n->next = NULL;
    return n;
}

int main(void) {
    struct Node *head = NULL, *tail = NULL;
    struct Node *temp, *a, *b, *prevA, *prevB;
    int n, value, k;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
```

```
for (int i = 0; i < n; i++) {
    scanf("%d", &value);
    temp = createNode(value);

    if (head == NULL)
        head = tail = temp;
    else {
        tail->next = temp;
        tail = temp;
    }
}

printf("Enter k: ");
scanf("%d", &k);

if (k < 1 || k > n) {
    printf("Invalid k\n");
    return 0;
}

int pos1 = k;
int pos2 = n - k + 1;

if (pos1 == pos2) {
    printf("Swapped list: ");
    for (temp = head; temp != NULL; temp = temp->next)
        printf("%d ", temp->data);
    printf("\n");
    return 0;
}

if (pos1 > pos2) {
    int t = pos1;
    pos1 = pos2;
    pos2 = t;
}

prevA = NULL;
a = head;

for (int i = 1; i < pos1; i++) {
    prevA = a;
    a = a->next;
}

prevB = NULL;
b = head;

for (int i = 1; i < pos2; i++) {
```

```
        prevB = b;
        b = b->next;
    }

    if (prevA != NULL)
        prevA->next = b;
    else
        head = b;

    if (prevB != NULL)
        prevB->next = a;
    else
        head = a;

    temp = a->next;
    a->next = b->next;
    b->next = temp;

    printf("Swapped list: ");

    for (temp = head; temp != NULL; temp = temp->next)
        printf("%d ", temp->data);

    printf("\n");
    return 0;
}
```

Output :

```
Enter number of nodes: 5
10 20 30 40 50
Enter k: 2
Swapped list: 10 40 30 20 50
```

10. Write a C program to multiply two numbers represented by two linked lists.

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int digit;
    struct Node *next;
};

struct Node *createNode(int digit) {
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    newNode->digit = digit;
```

```
    newNode->next = NULL;
    return newNode;
}

void insertEnd(struct Node **head, int digit) {
    struct Node *newNode = createNode(digit);

    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node *temp = *head;

    while (temp->next != NULL)
        temp = temp->next;

    temp->next = newNode;
}

int multiply(struct Node *a, struct Node *b) {
    int x = 0, y = 0;

    while (a != NULL) {
        x = x * 10 + a->digit;
        a = a->next;
    }

    while (b != NULL) {
        y = y * 10 + b->digit;
        b = b->next;
    }

    return x * y;
}

int main() {
    struct Node *a = NULL;
    struct Node *b = NULL;

    int n, m, digit;

    printf("Enter number of digits in first number: ");
    scanf("%d", &n);

    printf("Enter first number digits: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &digit);
        insertEnd(&a, digit);
    }
```

```
    printf("Enter number of digits in second number: ");
    scanf("%d", &m);

    printf("Enter second number digits: ");
    for (int i = 0; i < m; i++) {
        scanf("%d", &digit);
        insertEnd(&b, digit);
    }

    printf("Product: %d\n", multiply(a, b));

    return 0;
}
```

Output :

```
Enter number of digits in first number: 3
Enter first number digits: 1 0 0
Enter number of digits in second number: 2
Enter second number digits: 1 0
Product: 1000
```